Studyando

CICLO: Técnico Superior en Desarrollo de Aplicaciones Multiplataforma

Autor: Pedro José Martínez Manuel

Tutor/a: Abelardo León González

Convocatoria: Noviembre 2025/ Febrero 2026

Resumen
Con Studyando he buscado resolver el caos de tener la vida académica repartida en mil sitios. Es una aplicación para Android que unifica tres herramientas clave: un gestor de tareas, un horario por colores y un temporizador Pomodoro real. Para que la aplicación fuera fiable, la he construido con Kotlin y Room, asegurándome de que sea rápida y funcione. Más que una aplicación de productividad es un sistema pensado para que el estudiante baje sus niveles de estrés, deje de procrastinar y pueda llevar un control real de sus tiempos de estudio sin complicaciones técnicas.

1. Justificación. 2
2. Introducción. 3
3. Objetivos. 3
3.1. Objetivos Funcionales (Lo que el usuario ve) 3
3.2. Objetivos Técnicos (Lo que hay bajo la aplicación) 3
4. Desarrollo. 4
4.1. Stack Tecnológico 4
4.2. Arquitectura del Software 4
4.3 Estructura y Flujo de la Aplicación. 4
4.4. Persistencia y Gestión de Datos. 7
4.5. Módulo de Gestión de Tareas. 7
4.6. Módulo de Horario. 8
4.7. Módulo Pomodoro. 9
5. Conclusiones y Posibles Mejoras. 10
6. Bibliografía 11
7. Anexos 12

1. Justificación.

La idea detrás de Studyando es sencilla pero ambiciosa: dejar de saltar entre tres o cuatro aplicaciones diferentes para poder estudiar. Como estudiante, me di cuenta de que perdemos mucho tiempo y energía mental coordinando herramientas distintas: una para las tareas, otra para el horario y otra para medir los tiempos de estudio.

Quería unificar en un solo lugar los tres pilares que, bajo mi criterio, definen la productividad real:

- Organización: Para que nada se quede en el tintero y sepamos qué hay que hacer.
- Planificación: Para saber cuándo hacerlo mediante un horario visual organizado por colores.
- Ejecución: Para ponerse manos a la obra con un temporizador Pomodoro que de verdad funcione y no se detenga al salir de la pantalla.

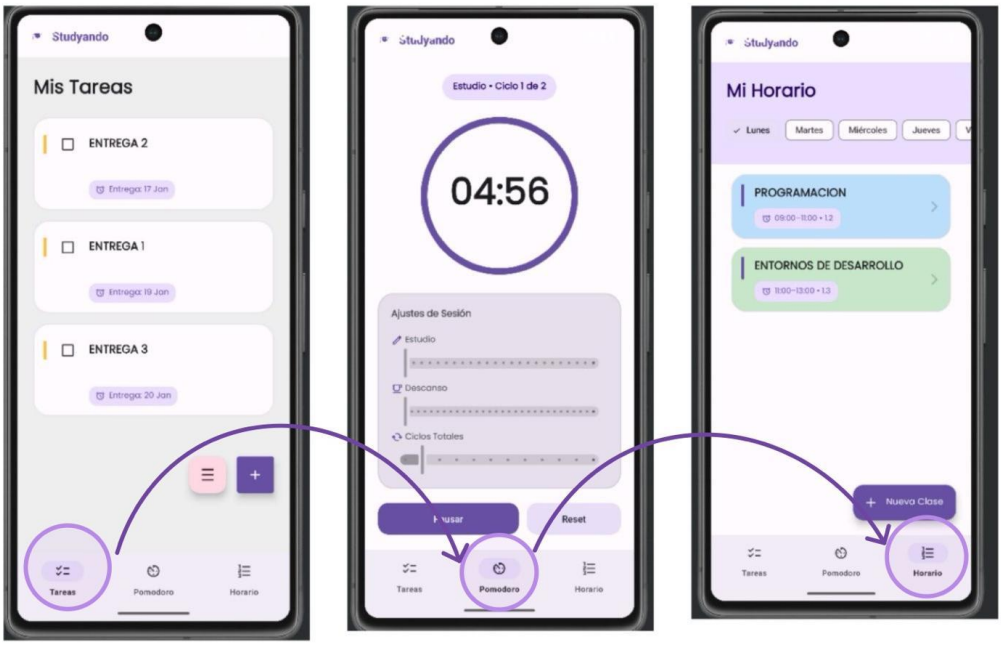


Figura 1. Las tres funcionalidades que tiene la aplicación (Tareas, Pomodoro y Horario).

2. Introducción.

Al final, el éxito de un estudiante no depende solo de lo inteligente que sea, sino de cómo administra su tiempo y su energía para no agobiarse. La fatiga mental es el mayor enemigo del rendimiento, y muchas

veces surge simplemente por no saber qué paso dar a continuación.

Por eso, decidí que la técnica Pomodoro debía ser el corazón de la aplicación. Para que Studyando fuera realmente útil, me centré en tres puntos clave que guiaron todo el desarrollo:

Diseño moderno: Utilicé Material Design 3 y la fuente Poppins. Buscaba una interfaz limpia, donde leer las tareas no cansara la vista y todo se sintiera familiar para un usuario de Android.

Arquitectura sólida: Implementé el patrón MVVM. Esto no es solo una decisión técnica; es lo que permite que la aplicación sea fluida, no se bloquee y responda al instante a lo que el usuario hace.

Experiencia de usuario natural: El flujo está diseñado para cerrar el ciclo de estudio. Desde que anotas una entrega, hasta que te sientas en tu escritorio y activas el contador para resolverla. Todo ocurre en el mismo ecosistema.

3. Objetivos.

La meta principal era crear una herramienta que acabara con el caos de la desorganización estudiantil. Para no perder el foco durante el desarrollo, dividí mis objetivos en dos frentes:

3.1. Objetivos Funcionales (Lo que el usuario ve)

- Gestión Centralizada: Permitir crear, editar y priorizar tareas académicas de forma intuitiva.
- Visualización Semanal: Un horario dinámico donde cada asignatura se identifique rápido por su color.
- Control de Tiempo: Un temporizador Pomodoro totalmente configurable y con alertas sonoras claras.

3.2. Objetivos Técnicos (Lo que hay bajo la aplicación)

- Integridad de Datos: Garantizar que la información nunca se pierda mediante persistencia local robusta.
- Multitarea Real: Lograr que el temporizador siga funcionando en segundo plano, incluso si el usuario bloquea el móvil.
- Código de Calidad: Crear una base de código escalable y limpia para que, en el futuro, añadir funciones nuevas sea sencillo y no rompa lo que ya funciona.

4. Desarrollo.

En esta sección detallo los procedimientos y técnicas que he llevado a cabo para resolver los problemas planteados.

4.1. Stack Tecnológico

Elegí estas tecnologías porque representan el estándar moderno de Google. Kotlin, es el lenguaje principal, lo elegí por su expresividad y porque evita muchos errores comunes (como los punteros nulos) que suelen dar problemas en Java.

- Corrutinas y Flow: Las utilizo para que las operaciones pesadas, como escribir en la base de datos, ocurran en hilos secundarios. Esto es vital, ya que, si la base de datos tarda medio segundo en guardar, el usuario no debería notar que la pantalla se congela.
- Dagger Hilt: Esta herramienta de inyección de dependencias me ayudó a mantener las piezas de la aplicación separadas pero conectadas de forma eficiente.
- Navigation Component: Para que el paso entre pantallas sea natural, con animaciones fluidas y sin errores de navegación.

4.2. Arquitectura del Software

Para Studyando me basé en el patrón MVVM.

- En Model es donde viven mis entidades de Room (como TaskEntity). Representado los datos puros.
- View: Son los fragmentos que el usuario toca. Su única función es mostrar información y avisar al cerebro (el ViewModel) cuando algo ocurre.
- ViewModel: Es el mediador. Lo mejor de usar ViewModels es que sobreviven a cambios de configuración, como rotar la pantalla. Si estás en medio de un Pomodoro y giras el teléfono, el tiempo sigue contando exactamente dónde estaba.

4.3 Estructura y Flujo de la Aplicación.

Para que Studyando sea una aplicación robusta y fácil de mantener, he diseñado una arquitectura dividida en tres capas claramente diferenciadas. Mi objetivo con este diseño es separar la cara visible de la aplicación de la lógica interna y el almacenamiento de datos, siguiendo las mejores prácticas de desarrollo en Android.

A continuación, detallo cómo interactúan estos componentes según el esquema técnico que he implementado:

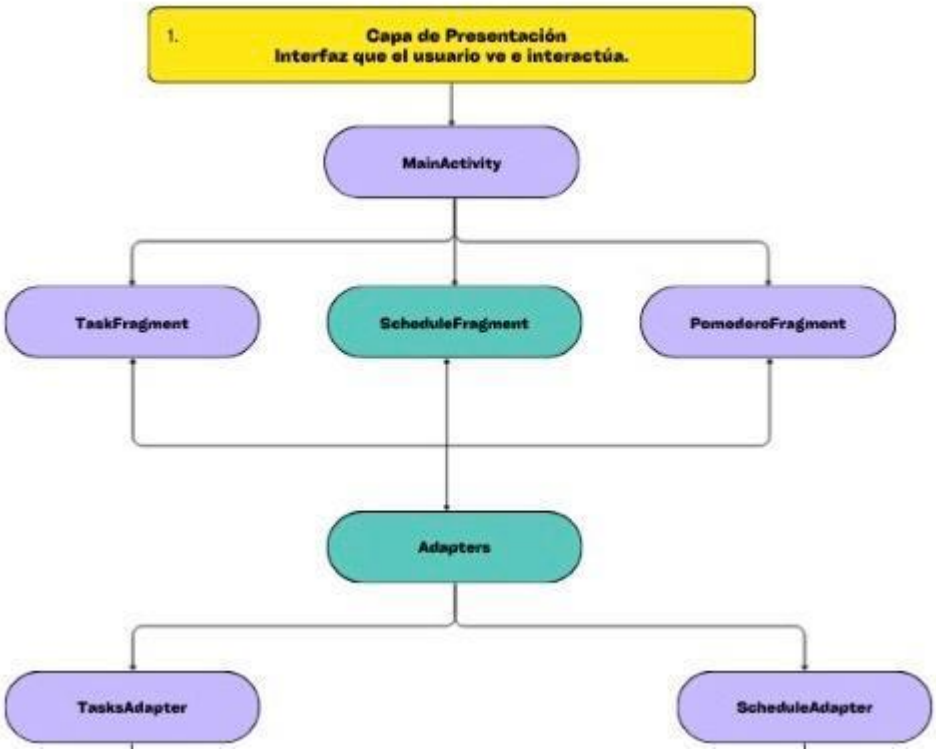


Figura 2. Diagrama de arquitectura de Studyando: flujo de datos de la capa Presentación.

Es la capa con la que el estudiante interactúa directamente. He centralizado todo el flujo en una MainActivity, que actúa como contenedor principal. De ella cuelgan los tres fragmentos clave de la app: TaskFragment (Tareas), ScheduleFragment (Horario) y PomodoroFragment.

Para que las listas de tareas y clases se vean de forma fluida, he implementado Adapters específicos (TasksAdapter y ScheduleAdapter). Estos se encargan de dibujar cada elemento en la pantalla de forma eficiente, asegurando que la navegación sea rápida y sin tirones visuales.

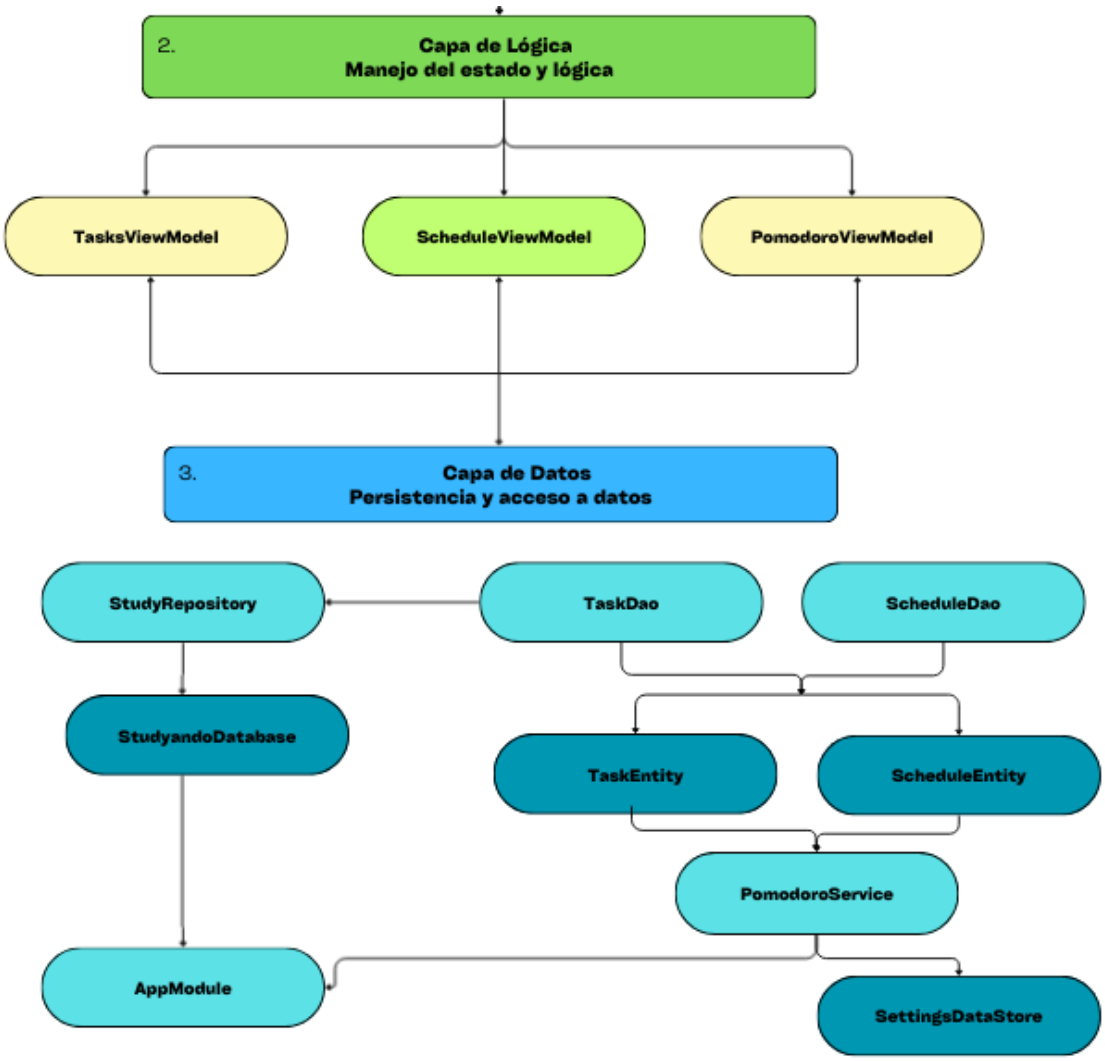


Figura 3. Diagrama de arquitectura de Studyando: flujo de datos de la capa Lógica y de Datos.

En la capa de Lógica es donde aplico el patrón MVVM. He creado un ViewModel para cada sección principal. Estos componentes son el cerebro de la interfaz; su función es procesar la información y servirla a los fragmentos. Una de las grandes ventajas de haberlo hecho así es que, si el usuario rota el teléfono o cambia de aplicación un momento, el estado (como el tiempo restante del Pomodoro o la lista filtrada de tareas) no se pierde.

La capa de datos la base de la pirámide y donde reside la seguridad de la información. He implementado un StudyRepository que actúa como mediador: los ViewModels no hablan directamente con la base de datos, sino que le piden los datos al repositorio.

- Persistencia Relacional: Uso Room con sus DAOs (TaskDao, ScheduleDao) para manejar las tareas y el horario de forma estructurada.

- Preferencias: Para ajustes ligeros, como la configuración de minutos del Pomodoro, utilizo SettingsDataStore.
- Procesos Especiales: En esta capa también vive el PomodoroService, que permite que el contador siga vivo en segundo plano.

4.4. Persistencia y Gestión de Datos.

Un estudiante no puede permitirse perder su horario justo antes de un examen. Por eso, puse especial cuidado en cómo se guardan los datos.

Utilizo Room como una capa sobre SQLite. Es robusto y me permite hacer consultas complejas de forma segura. Durante el desarrollo, tuve que actualizar la base de datos (migración a la versión 2) para incluir colores personalizados en las asignaturas, un reto técnico que resolví configurando migraciones destructivas controladas para no interrumpir el flujo de trabajo.

Para cosas pequeñas, como cuánto tiempo quieres que dure tu sesión de Pomodoro (ej. 25 o 50 minutos), no necesitaba una tabla entera. Usé DataStore, que es mucho más moderno y seguro que el antiguo SharedPreferences, ya que trabaja de forma asíncrona.

4.5. Módulo de Gestión de Tareas.

Este es el motor de la organización. No quería una lista plana de texto, quería algo que ayudara a priorizar.

- Prioridades: Cada tarea tiene un nivel (Alta, Media, Baja) que el usuario puede ver de un vistazo.
- Validación de Fechas: Si una tarea vence y el usuario no la ha marcado como completada, la aplicación cambia el color de la fecha a rojo automáticamente. Es un pequeño detalle visual que genera urgencia.
- Interacción Humana: Implementé el gesto de deslizar para borrar (Swipe-to-delete). Si borras algo por error, aparece un mensaje rápido ("Undo") para recuperarlo al instante. Esto hace que la aplicación se sienta más viva y perdonadora con los errores del usuario.
- Registro rápido y detallado: Mediante un botón flotante, el usuario puede añadir una tarea asignándole un título, notas adicionales y una fecha de entrega.
- Además, hay otro botón flotante donde se van guardando cada una de las tareas marcadas como realizadas.

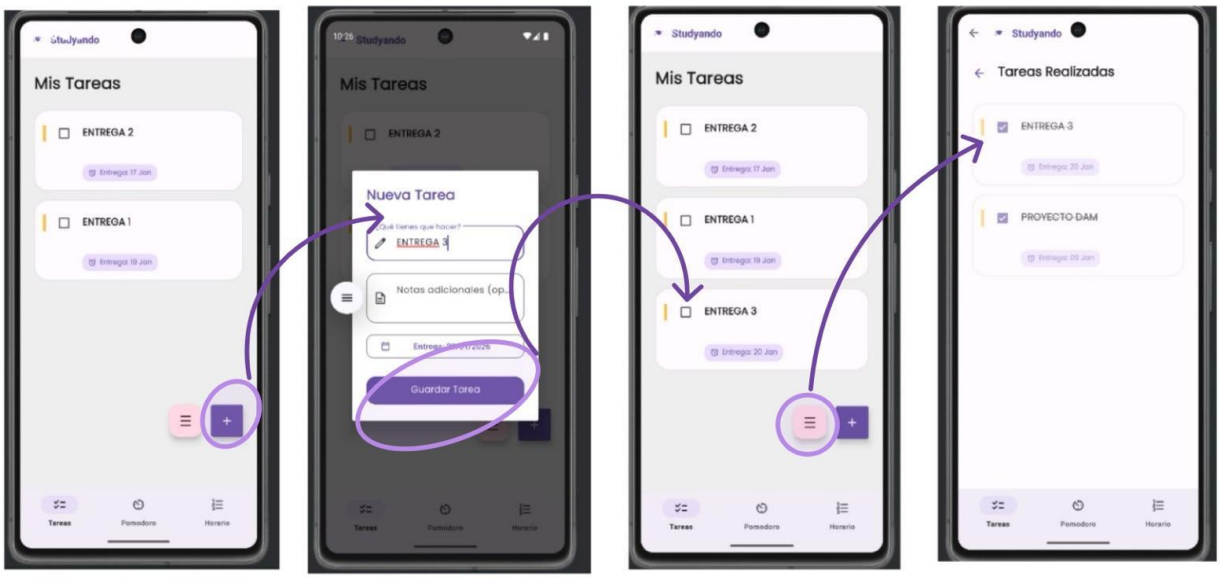


Figura 4. Demostración de funcionamiento de la parte de crear tareas y ver las realizadas.

4.6. Módulo de Horario.

El horario está diseñado para la rapidez mental. En lugar de una lista interminable, el usuario usa "Chips" para saltar entre los días de la semana.

- Identidad Visual: Cada clase tiene un color asignado. Al crear una asignatura, el usuario elige un tono de una paleta que yo genero dinámicamente.
- Sincronización: Gracias al uso de Flow, en cuanto añades una clase en el diálogo emergente, el horario de fondo se actualiza solo, sin necesidad de botones de "refrescar".
- Detalles de ubicación: Cada entrada del horario permite registrar no solo la hora, sino el aula o el enlace a la clase online, centralizando toda la información logística de la jornada.

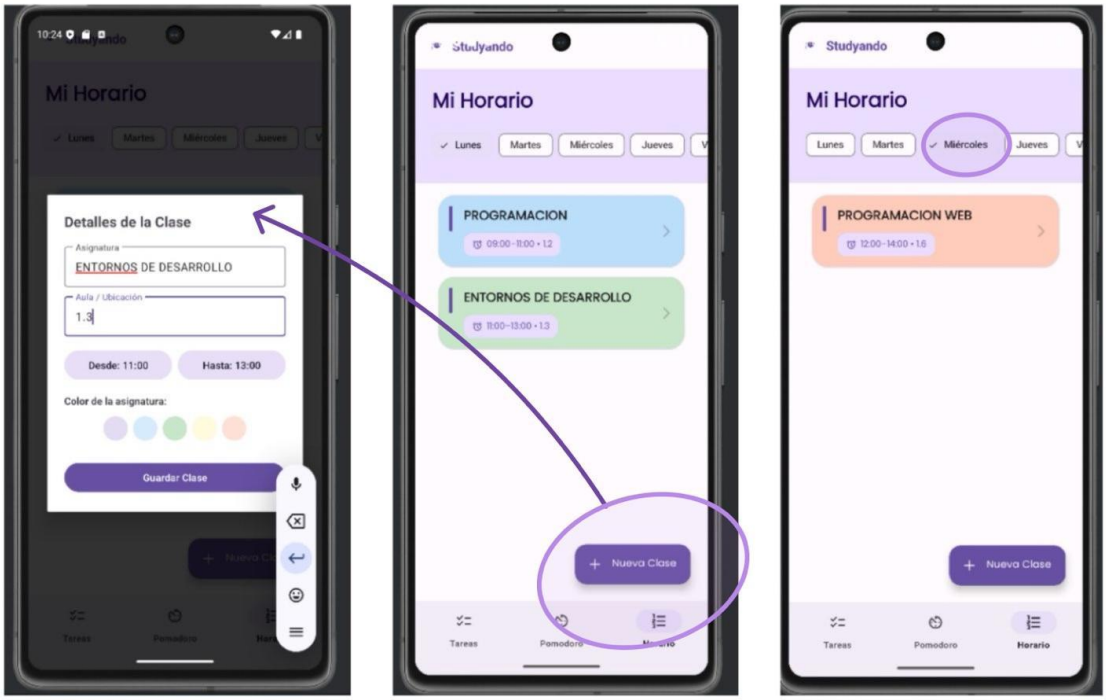


Figura 5. Demostración de funcionamiento a la hora de crear clases y añadirlas al horario, además de ver en la tercera imagen la selección de otro día de la semana con sus clases correspondientes.

4.7. Módulo Pomodoro.

Este es el módulo donde el estudiante realmente se pone a trabajar. Las funcionalidades clave que he desarrollado son:

- Configuración de sesiones: A través de la persistencia con DataStore, el usuario puede elegir cuánto tiempo quiere concentrarse y cuánto descansar, adaptando la herramienta a su propio ritmo.
- Automatización de ciclos: La aplicación gestiona por sí sola las transiciones. Cuando el tiempo de estudio termina, suena una alerta y comienza el descanso sin que el usuario tenga que tocar el móvil.
- Trabajo ininterrumpido en segundo plano: Esta es la función de la que más orgulloso estoy. Gracias al Foreground Service, el estudiante puede salir de la app para consultar un PDF o buscar información en el navegador mientras el tiempo sigue corriendo.
- Feedback acústico: He integrado el sistema de notificaciones de Android para que el móvil avise con un sonido al cambiar de fase, permitiendo que el usuario se aleje de la pantalla durante los descansos.

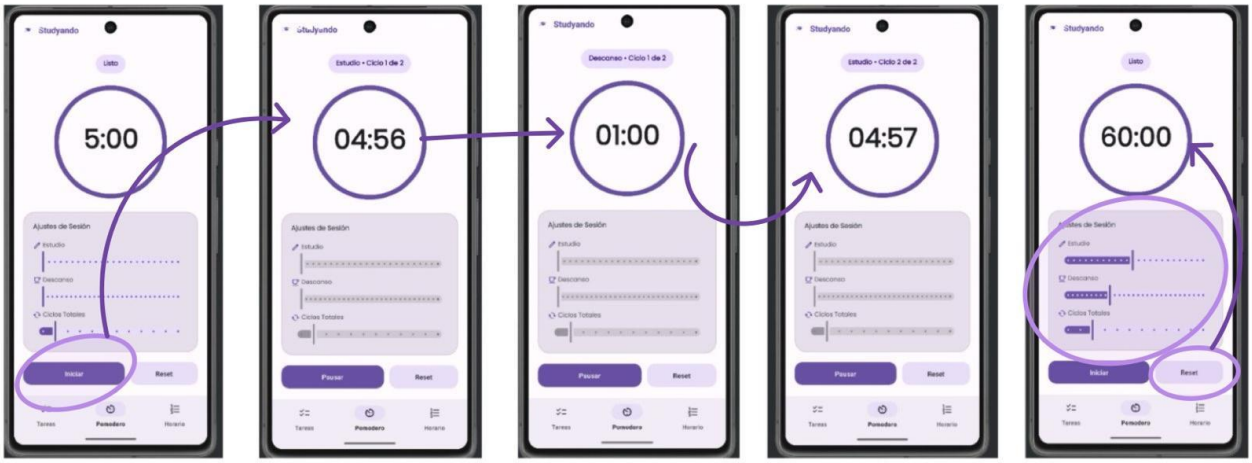


Figura 6. Funcionamiento del método pomodoro donde establecemos un tiempo de estudio, descanso y ciclos y le damos a Iniciar. Vemos cómo cambia a la fase descanso al terminar la de estudio y empieza un nuevo ciclo al terminar la de descanso y por último en la última pantalla se observa que estableciendo nuevos parámetros y dándole a "Reset" ponemos otros tiempos para el método.

5. Conclusiones y Posibles Mejoras.

La culminación de Studyando representa mucho más que el simple desarrollo de una aplicación es la respuesta a una necesidad real de organización que detecté en el entorno académico. Tras finalizar el proceso, he llegado a varias conclusiones clave que validan tanto el esfuerzo técnico como el diseño funcional del proyecto.

Si tengo que ser sincero, pelearme con la arquitectura MVVM y la persistencia de Room fue uno de los mayores desafíos de mi formación. Pero esa dificultad es la que me ha hecho crecer como futuro técnico superior. Entender cómo fluyen los datos y ver que la interfaz reacciona al instante a lo que el usuario hace, me ha dado una confianza profesional que no tenía antes de empezar esta memoria.

Como desarrollador de este ecosistema, no veo Studyando como un proyecto cerrado, sino como una semilla que puede seguir creciendo. Si tuviera que continuar trabajando en ella mañana mismo, estas serían mis prioridades:

- Sincronización total: Me encantaría implementar una base de datos en la nube para que mis tareas me acompañen del móvil a la tablet sin esfuerzo.

- Motivación mediante recompensas: Creo firmemente que estudiar puede ser más divertido. Mi idea es añadir un sistema de "logros" que premie al estudiante por cada sesión de Pomodoro completada con éxito.

- Conexión con mi música: Como usuario, sé que la música es clave para concentrarse. Por eso, una mejora prioritaria sería integrar Spotify directamente en el temporizador para no tener que salir de la aplicación para cambiar de canción.

- Accesibilidad para todos: Siento que la tecnología debe ser inclusiva. Por eso, quiero trabajar en un Modo Oscuro más avanzado y en mejorar el soporte para lectores de pantalla, asegurando que cualquier estudiante pueda organizar su vida con mi aplicación sin barreras.

6. Bibliografía

En cuanto a la documentación técnica aquí detallo las fuentes de información y recursos técnicos que han servido de base para el diseño, desarrollo y documentación de Studyando. He recurrido principalmente a documentación oficial para asegurar que la aplicación cumpla con los estándares actuales de desarrollo en Android.

Documentación técnica y Frameworks.

- Android Developers. Guía de arquitectura de aplicaciones móviles. Buscado en: https://developer.android.com/topic/architecture.

- Android Developers. Persistencia de datos con Room. Buscado en: https://developer.android.com/training/data-storage/room.

- Android Developers. Servicios de primer plano (Foreground Services). Buscado en: https://developer.android.com/guide/components/foreground-services.

- Kotlin Documentation. Corrutinas de Kotlin: Fundamentos y mejores prácticas. Buscado en: https://kotlinlang.org/docs/coroutines-overview.html.

- Google Developers. Guía de Jetpack DataStore para almacenamiento de preferencias. Buscado en: https://developer.android.com/topic/libraries/architecture/datastore.

Diseño de Interfaz y Experiencia de Usuario.

- Google Material Design. Material Design 3: Material You. Buscado en: https://m3.material.io/.

Patrones de Diseño e Inyección de Dependencias.

- Dagger Hilt Official Guide. Inyección de dependencias en Android con Hilt. Buscado en:

https://developer.android.com/training/dependency-injection/hilt-android.

- Keep Coding. Patrón MVVM. https://keepcoding.io/blog/pasos-para-la-implementacion-de-mvvm-en-android/

7. Anexos

En esta sección incluyo los fragmentos de código que considero el "motor" de Studyando. He seleccionado estas clases porque representan los mayores desafíos técnicos resueltos: la reactividad de los datos, la inyección de dependencias y la ejecución en segundo plano.

- Anexo A: Núcleo Técnico (Fragmentos de Código Fuente)

La gestión de tareas se apoya en un Objeto de Acceso a Datos (DAO) que utiliza Flow. Esto permite que la interfaz de usuario se actualice automáticamente sin que yo tenga que programar refrescos manuales.

```
7 Usages  1 Implementation
@Dao
interface TaskDao {
    1 Usage  1 Implementation
    @Query( value = "SELECT * FROM tasks")
    fun getAllTasks(): Flow<List<TaskEntity>>

    1 Usage  1 Implementation
    @Query( value = "SELECT * FROM tasks WHERE isDone = 0 ORDER BY priority, COALESCE(dueAt, 9223372036854775807)")
    fun observeOpenTasks(): Flow<List<TaskEntity>>

    1 Usage  1 Implementation
    @Query( value = "SELECT * FROM tasks WHERE isDone = 1 ORDER BY dueAt DESC")
    fun observeCompletedTasks(): Flow<List<TaskEntity>>

    1 Usage  1 Implementation
    @Insert suspend fun insert(task: TaskEntity): Long
    2 Usages  1 Implementation
    @Update suspend fun update(task: TaskEntity)
    1 Usage  1 Implementation
    @Delete suspend fun delete(task: TaskEntity)

    1 Implementation
    @Query( value = "UPDATE tasks SET isDone = 1 WHERE id = :id")
    suspend fun markDone(id: Long)
}
```

Figura 7. Código de TaskDao que aporta persistencia de Datos y Consultas Reactivas.

- Anexo B: Grafo de navegación.

```xml
<?xml version="1.0" encoding="utf-8"?>
<navigation xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/nav_graph"
    app:startDestination="@id/scheduleFragment">

    <fragment
        android:id="@+id/tasksFragment"
        android:name="com.studyando.ui.tasks.TasksFragment"
        android:label="Tareas"
        tools:layout="@layout/fragment_tasks">
        <action
            android:id="@+id/action_tasksFragment_to_addEditTaskFragment"
            app:destination="@id/addEditTaskFragment" />
        <action
            android:id="@+id/action_tasksFragment_to_completedFragment"
            app:destination="@id/completedFragment" />
    </fragment>

    <fragment
        android:id="@+id/pomodoroFragment"
        android:name="com.studyando.ui.pomodoro.PomodoroFragment"
        android:label="Pomodoro" />

    <fragment
        android:id="@+id/completedFragment"
        android:name="com.studyando.ui.completed.CompletedFragment"
        android:label="Realizadas" />

    <fragment
        android:id="@+id/scheduleFragment"
        android:name="com.studyando.ui.schedule.ScheduleFragment"
        android:label="Horario" />

    <dialog
        android:id="@+id/addEditTaskFragment"
        android:name="com.studyando.ui.tasks.AddEditTaskFragment"
        android:label="Añadir Tarea" >
        <argument
            android:name="taskId"
            app:argType="long"
            android:defaultValue="-1L" />
    </dialog>

</navigation>
```

Figura 7. Código de en él se conectan los fragmentos programados.

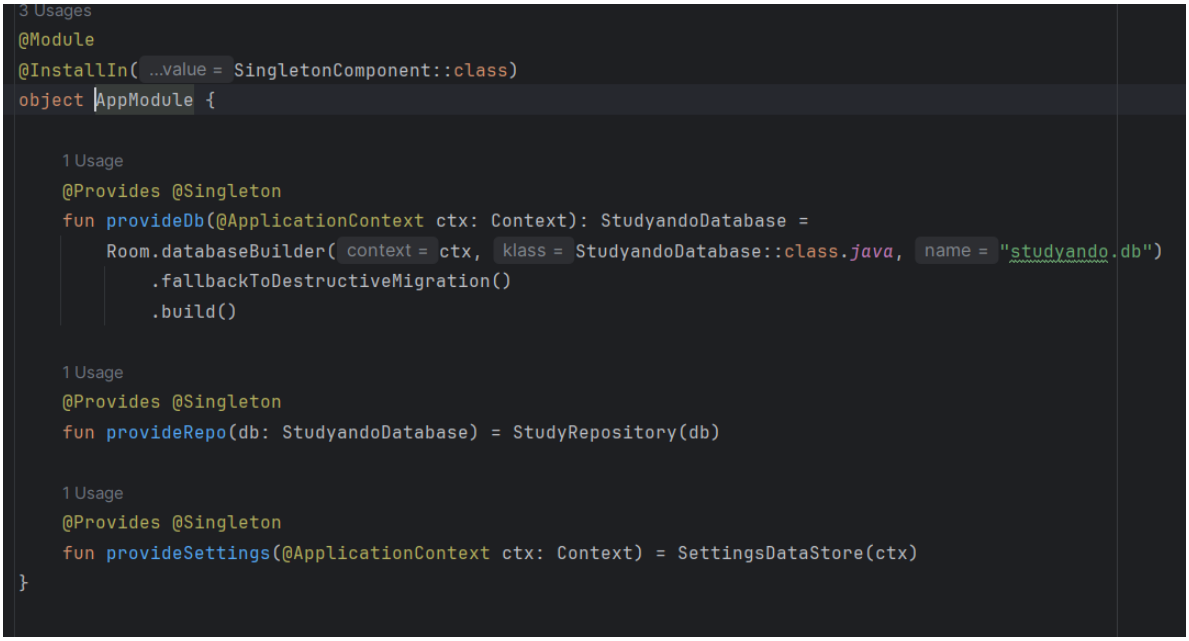
- Anexo C: Filtrado por días.

```
1 Usage
private fun initChips() {
    binding.chipGroupDays.setOnCheckedStateChangeListener { _, ids ->
        if (ids.isNotEmpty()) {
            selectedDay = when (ids.first()) {
                R.id.chipMon -> 1
                R.id.chipTue -> 2
                R.id.chipWed -> 3
                R.id.chipThu -> 4
                R.id.chipFri -> 5
                R.id.chipSat -> 6
                R.id.chipSun -> 7
                else -> 1
            }
            refreshData(selectedDay)
        }
    }
    binding.chipMon.isChecked = true
    refreshData( day = 1)
}
```

Figura 8. Código de en él se filtran los días de la semana en el método de Horario (ScheduleFragment).

- Anexo D: Inyección de Dependencias.

El archivo AppModule.kt es el pegamento de la arquitectura.



```
3 Usages
@Module
@InstallIn( ...value = SingletonComponent::class)
object AppModule {

    1 Usage
    @Provides @Singleton
    fun provideDb(@ApplicationContext ctx: Context): StudyandoDatabase =
        Room.databaseBuilder( context = ctx, klass = StudyandoDatabase::class.java, name = "studyando.db")
            .fallbackToDestructiveMigration()
            .build()

    1 Usage
    @Provides @Singleton
    fun provideRepo(db: StudyandoDatabase) = StudyRepository(db)

    1 Usage
    @Provides @Singleton
    fun provideSettings(@ApplicationContext ctx: Context) = SettingsDataStore(ctx)
}
```

Figura 9. Código en el que se establecen las dependencias de la aplicación.